Tipos de JOIN

Unir tablas paso a paso, con ejemplos sencillos

Semana 09 - Sesión 17

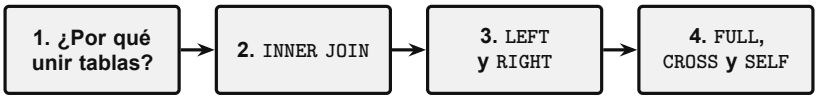
Rel Guzman



Logro de la sesión

Al finalizar, podrás **unir dos tablas** con `JOIN` y **elegir el tipo de JOIN** según lo que quieras conservar.

Lo de hoy: cuatro paradas



- Ya sabes consultar **una tabla**. Hoy consultamos **dos a la vez**.
- Usaremos **las mismas dos tablas** todo el tiempo: solo cambia el tipo de `JOIN`.

Ojo con esto

Vamos **de a pocos**: una idea por diapositiva y una práctica después de cada parada.

Unir tablas

Los datos viven en tablas separadas

`clientes`

id	nombre
1	Ana
2	Luis
3	María

`pedidos`

id	producto	cliente_id
101	Laptop	1
102	Mouse	1
103	Teclado	2
104	Monitor	*NULL*

Pregunta: ¿quién pidió el **Teclado**?

1. En `pedidos`: el Teclado tiene `cliente_id` = **2**.
2. En `clientes`: el `id` **2** es **Luis**.

La idea

Tú lo resolviste **mirando dos tablas** y juntando lo que coincide. Eso mismo hace un `JOIN`, en **una sola consulta**.

En palabras simples

Tienes una hoja con **códigos y notas**, y otra con **códigos y nombres**. Las pones lado a lado para saber de quién es cada nota.

Un `JOIN`, pieza por pieza

```
SELECT c.nombre, p.producto
FROM clientes c
JOIN pedidos p
ON p.cliente_id = c.id;
```

◀ qué columnas quiero, y **de qué tabla**

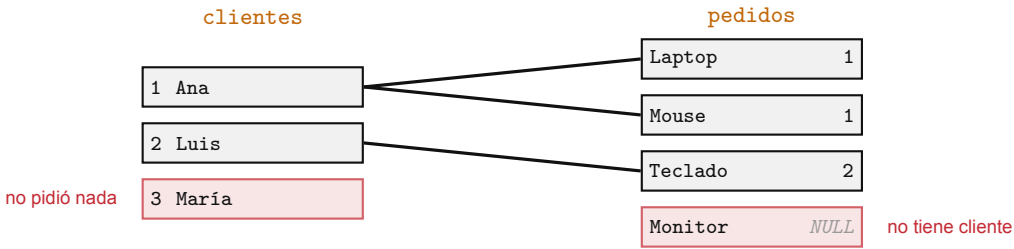
◀ la tabla de la **izquierda**: se escribe primero

◀ la tabla de la **derecha**: se escribe después

◀ cómo se **emparejan** las filas

- `c` y `p` son **apodos** (alias) de las tablas: ahorran escribir el nombre completo.
- `c.nombre` se lee «la columna `nombre` de `clientes`».
- El `ON` casi siempre compara la **clave foránea** con la **clave primaria**.

Al emparejar, algunos se quedan sin pareja



- **Con pareja:** Ana (dos pedidos) y Luis (un pedido).
- **Sin pareja:** María y el Monitor.

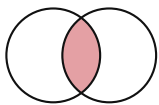
La única pregunta de hoy

Cada tipo de `JOIN` responde lo mismo: **¿qué hago con los que se quedaron sin pareja?**

INNER JOIN

`INNER JOIN`: solo los que tienen pareja

```sql
SELECT c.nombre, p.producto
FROM clientes c
INNER JOIN pedidos p ON p.cliente_id = c.id;
```



solo lo que está en **las dos** tablas

resultado

nombre	producto
Ana	Laptop
Ana	Mouse
Luis	Teclado

- Los que no tienen pareja **quedan fuera**: no aparecen ni María ni el Monitor.
- Ana aparece **dos veces**: una fila por cada pedido suyo.
- Escribir solo `JOIN` es lo mismo que `INNER JOIN`. Es el más usado.

En palabras simples

Es una **fiesta solo para parejas**: el que llega solo, no entra.

Práctica 1 INNER JOIN en la universidad

carreras

id	nombre
1	Sistemas
2	Industrial
3	Civil

alumnos

id	nombre	carrera_id
1	Paola	1
2	Jorge	1
3	Inés	3
4	Tomás	*NULL*

```sql
SELECT c.nombre, a.nombre
FROM carreras c
INNER JOIN alumnos a
   ON a.carrera_id = c.id;
```

Pregunta	Respuesta
¿Cuántas filas tiene el resultado?	_____
¿Qué carrera **no** aparece? ¿Por qué?	_____
¿Qué alumno **no** aparece? ¿Por qué?	_____

Para escribir: anota en tu cuaderno las filas del resultado, una por una.

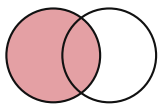
LEFT y RIGHT

`LEFT JOIN`: toda la tabla izquierda

```sql
SELECT c.nombre, p.producto
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id;
```

resultado

nombre	producto
Ana	Laptop
Ana	Mouse
Luis	Teclado
María	*NULL*



toda la izquierda, tenga pareja o no

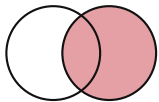
- La **izquierda** es la tabla del `FROM`: `clientes`. **Ninguno** de sus clientes se pierde.
- María no tiene pedido: su `producto` se rellena con *NULL*.
- El Monitor sigue fuera: está en la tabla de la **derecha**.

En palabras simples

Es la **lista de asistencia**: salen todos los alumnos, y al que faltó se le deja el casillero vacío.

RIGHT JOIN: toda la tabla derecha

```sql
SELECT c.nombre, p.producto
FROM clientes c
RIGHT JOIN pedidos p ON p.cliente_id = c.id;
```



toda la derecha, tenga pareja o no

resultado

nombre	producto
Ana	Laptop
Ana	Mouse
Luis	Teclado
NULL	Monitor

- La **derecha** es la tabla del `JOIN`: `pedidos`. **Ningún** pedido se pierde.
- El Monitor no tiene cliente: su `nombre` se rellena con *NULL*.
- Ahora la que queda fuera es María.

Es el **mismo** `LEFT JOIN`, visto desde el otro lado.

LEFT y RIGHT: lo mismo, al revés

Con RIGHT

```
FROM clientes c
RIGHT JOIN pedidos p
   ON p.cliente_id = c.id;
```

Con LEFT

```
FROM pedidos p
LEFT JOIN clientes c
   ON p.cliente_id = c.id;
```

Las dos conservan **todos los pedidos**. Solo cambió **el orden** en que se escriben las tablas.

- **Truco:** pregúntate «¿de qué tabla **no quiero perder** ninguna fila?».
- Pon esa tabla en el `FROM` y usa `LEFT JOIN`.

Cómo se ve en la realidad

Casi todos escriben siempre `LEFT JOIN` y acomodan el orden de las tablas. El `RIGHT JOIN` se ve muy poco.

Un clásico: encontrar a los que no tienen pareja

Pregunta: ¿qué clientes **nunca** han hecho un pedido?

```sql
SELECT c.nombre
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

resultado

nombre
María

1. El `LEFT JOIN` trae a **todos** los clientes; a María le pone *NULL* en el pedido.
2. El `WHERE` se queda **solo con las filas que tienen ese *NULL***.

Cómo se ve en la realidad

Así se buscan los alumnos **sin matrícula**, los productos **que nadie compró** o los empleados **sin área**.

Práctica 2 LEFT y RIGHT en la universidad

carreras

id	nombre
1	Sistemas
2	Industrial
3	Civil

alumnos

id	nombre	carrera_id
1	Paola	1
2	Jorge	1
3	Inés	3
4	Tomás	*NULL*

```
SELECT c.nombre, a.nombre
FROM carreras c
_________ JOIN alumnos a
   ON a.carrera_id = c.id;
```

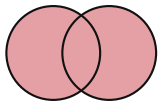
	Con LEFT	Con RIGHT
¿Cuántas filas salen?		
¿Qué fila lleva un *NULL*?		
¿Quién queda fuera?		

Para escribir: la consulta que muestra las carreras que **no tienen alumnos**.

FULL, CROSS y SELF

FULL OUTER JOIN: nadie queda fuera

```sql
SELECT c.nombre, p.producto
FROM clientes c
FULL OUTER JOIN pedidos p ON p.cliente_id = c.id;
```



todo lo de las dos tablas

resultado

nombre	producto
Ana	Laptop
Ana	Mouse
Luis	Teclado
María	*NULL*
NULL	Monitor

- Es un `LEFT` y un `RIGHT` **juntos**: aparecen María **y** el Monitor.
- A cada uno se le rellena con *NULL* el lado que le falta.

Ojo con esto

MySQL no tiene `FULL OUTER JOIN`. Ahí se arma juntando un `LEFT JOIN` y un `RIGHT JOIN` con `UNION`.

`CROSS JOIN`: todos con todos

`tallas`

talla
S
M

`colores`

color
rojo
azul
negro

```sql
SELECT t.talla, c.color
FROM tallas t
CROSS JOIN colores c;
```

talla	color
S	rojo
S	azul
S	negro
M	rojo
M	azul
M	negro

- **No lleva** `ON`: no empareja nada, combina **cada fila con todas** las de la otra tabla.
- Las filas se **multiplican**: 2 tallas $\times$ 3 colores = **6 filas**.

Ojo con esto

Con tablas grandes explota: 1 000 $\times$ 1 000 = **1 000 000** de filas. Se usa poco, y a propósito.

SELF JOIN: una tabla unida consigo misma

empleados

id	nombre	jefe_id
1	Rosa	*NULL*
2	Carlos	1
3	Elena	1
4	Diego	2

```sql
SELECT e.nombre AS empleado,
       j.nombre AS jefe
FROM empleados e
JOIN empleados j ON e.jefe_id = j.id;
```

empleado	jefe
Carlos	Rosa
Elena	Rosa
Diego	Carlos

- El jefe **también es un empleado**: está en la misma tabla.
- **No es una palabra nueva**: es un `JOIN` de siempre, con la tabla escrita **dos veces**.
- Los apodos son **obligatorios**: `e` es el empleado y `j` es su jefe.

Rosa no aparece como empleado porque no tiene jefe. ¿Con qué `JOIN` sí aparecería?

Todos los `JOIN` en una tabla

	Tipo	Qué conserva	Filas
	`INNER JOIN`	solo los que tienen pareja	3
	`LEFT JOIN`	toda la izquierda	4
	`RIGHT JOIN`	toda la derecha	4
	`FULL OUTER JOIN`	todo, de los dos lados	5
×	`CROSS JOIN`	todas las combinaciones, sin `ON`	12

Las filas son las de `clientes` (3) con `pedidos` (4). El `SELF JOIN` no está: es cualquiera de estos, con la misma tabla dos veces.

¿Qué `JOIN` usarías?

Lo que piden	`JOIN`
Cada alumno con su carrera; **solo** los que tienen carrera.	_____
Todas las carreras, tengan alumnos o no.	_____
Carreras y alumnos, **sin dejar a nadie** fuera.	_____
Todas las combinaciones de 3 polos y 4 colores.	_____
Cada empleado con el nombre de **su jefe**.	_____

- ¿Cuántas filas salen en la cuarta? _____
- Con `carreras` (3) y `alumnos` (4), ¿cuántas filas da el `FULL OUTER JOIN`? _____

Preguntas



Cierre: lo de hoy en una tabla

	En una frase
`JOIN`	Junta filas de dos tablas que coinciden en el `ON`.
`INNER`	Solo los que tienen pareja.
`LEFT` / `RIGHT`	Toda una tabla; lo que falta del otro lado queda en *NULL*.
`FULL` / `CROSS` / `SELF`	Todo de ambas; todos con todos; la tabla consigo misma.

La pregunta para elegir un `JOIN`: **¿qué hago con los que no tienen pareja?**

Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.

Oracle. (s. f.). *MySQL reference manual: JOIN clause*. `https://dev.mysql.com/doc/refman/8.4/en/join.html`



UTP Universidad Tecnológica del Perú